Complementary Filter

1.0

Generated by Doxygen 1.17.0

Chapter 1

Complementary Filter Library

1.1 Introduction

This library provides a generic implementation of a first-order complementary filter for sensor fusion.

A complementary filter combines measurements from different sensors by exploiting their complementary frequency characteristics. A low-frequency measurement from one sensor can be combined with a high-frequency measurement from another sensor to obtain a more stable estimate.

The library provides a templated estimators::cf::CF class that supports different numeric types such as `float` and `double`.

1.2 Features

The complementary filter library provides:

- First-order complementary filtering.
- Fusion of two sensor measurements.
- Configurable cutoff frequency.
- Configurable sampling time.
- Configurable filter time constant.
- Configurable complementary filter coefficient.
- Runtime parameter modification.
- Filter state reset functionality.
- Support for different numeric data types through templates.

1.3 Basic Usage

A complementary filter can be created and initialized using a cutoff frequency and sampling time:

```
#include "cf.h"

estimators::cf::CF<double> filter;

filter.init(2.5, 0.006);

double output = filter.update(x1, x2);
```

1.4 Filter Parameters

The filter can be configured using the following parameters:

- `fc` : Cutoff frequency of the filter.
- `tau` : Time constant of the filter.
- `dt` : Sampling time or control-loop period.
- `alpha`: Complementary filter coefficient.

The parameters can be configured during initialization or individually using the corresponding setter functions.

1.5 Parameter Configuration

The filter can be initialized using:

```
filter.init(fc, dt);
```

Alternatively, individual parameters can be configured using:

```
filter.set_fc(fc);
filter.set_tau(tau);
filter.set_dt(dt);
filter.set_alpha(alpha);
```

1.6 Updating the Filter

New sensor measurements are provided to the filter using the estimators::cf::CF::update function:

```
double output = filter.update(x1, x2);
```

The two input measurements are combined according to the configured complementary filter parameters, and the filtered output is returned.

1.7 Resetting the Filter

The internal filter state can be reset using:

```
filter.reset();
```

1.8 Accessing Parameters

The current filter parameters can be obtained using:

```
double fc    = filter.get_fc();
double tau   = filter.get_tau();
double dt    = filter.get_dt();
double alpha = filter.get_alpha();
```

1.9 Main Class

The main class provided by this library is:

- estimators::cf::CF - Generic complementary filter for sensor fusion.

1.10 Namespace

The complementary filter implementation is contained in:

```
estimators::cf
```

1.11 Complete Example

```
#include "cf.h"

int main()
{
    estimators::cf::CF<double> filter;

    filter.init(2.5, 0.006);

    double x1 = 10.0;
    double x2 = 10.5;

    double output = filter.update(x1, x2);

    return 0;
}
```

1.12 Detailed Documentation

Detailed documentation for the complementary filter class and its member functions is available through the estimators::cf::CF class reference.

Copyright

Copyright (c) 2026 Abhinay Kumar

Author

Abhinay Kumar

Version

1.0

Date

2026-08-11

Chapter 2

Complementary_filter

Complementary_filter

Chapter 3

Directory Hierarchy

3.1 Directories

Chapter 4

Namespace Index

4.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 6

File Index

6.1 File List

Here is a list of all files with brief descriptions:

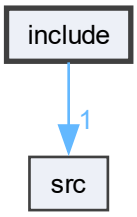
Chapter 7

Directory Documentation

7.1 doc Directory Reference

7.2 include Directory Reference

Directory dependency graph for include:



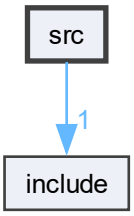
Files

- file cf.h

  Declaration of the complementary filter class.

7.3 src Directory Reference

Directory dependency graph for src:



Files

- file cf.tpp

Chapter 8

Namespace Documentation

8.1 cf Namespace Reference

Contains the complementary filter implementation.

8.1.1 Detailed Description

Contains the complementary filter implementation.

8.2 estimators Namespace Reference

Contains estimation and sensor-fusion algorithms.

Namespaces

- namespace cf

8.2.1 Detailed Description

Contains estimation and sensor-fusion algorithms.

8.3 estimators::cf Namespace Reference

Classes

- class CF

  Generic complementary filter for sensor fusion.

Chapter 9

Class Documentation

9.1 estimators::cf::CF< T > Class Template Reference

Generic complementary filter for sensor fusion.

`#include <cf.h>`

Public Member Functions

- CF ()
  Constructs a complementary filter.
- ~CF ()
  Destroys the complementary filter.
- T get_alpha ()
  Gets the complementary filter coefficient.
- T get_dt ()
  Gets the sampling time.
- T get_fc ()
  Gets the cutoff frequency.
- T get_tau ()
  Gets the filter time constant.
- void init (T fc_, T dt_)
  Initializes the complementary filter.
- void reset ()
  Resets the filter state.
- void set_alpha (T alpha_)
  Sets the complementary filter coefficient.
- void set_dt (T dt_)
  Sets the sampling time.
- void set_fc (T fc_)
  Sets the cutoff frequency.
- void set_param (T fc_, T dt_)
  Sets the filter parameters.
- void set_tau (T tau_)
  Sets the filter time constant.
- T update (T x1_i, T x2_i)
  Updates the complementary filter with new measurements.

Private Attributes

- T alpha = 0.0

  Complementary filter coefficient.
- T dt = 0.0

  Sampling time or control-loop period.
- T fc = 0.0

  Cutoff frequency of the filter.
- T tau = 0.0

  Time constant of the filter.

9.1.1 Detailed Description

template<typename T>
class estimators::cf::CF< T >

Generic complementary filter for sensor fusion.

The CF class implements a first-order complementary filter that combines two input measurements into a single filtered output.

The filter can be configured using either the cutoff frequency and sampling time or directly using the filter parameters.

The class is templated so that the filter can operate on different numeric types such as `float` or `double`.

Template Parameters

T	Numeric data type used for the filter calculations.

Typical usage

```
estimators::cf::CF<double> filter;

filter.init(2.5, 0.006);

double output = filter.update(x1, x2);
```

Definition at line 69 of file cf.h.

9.1.2 Constructor & Destructor Documentation

9.1.2.1 CF()

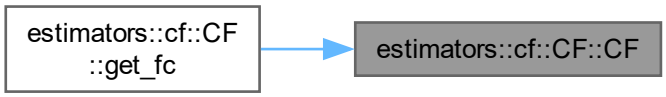
```
template<typename T>
estimators::cf::CF< T >::CF ()
```

Constructs a complementary filter.

The filter parameters are initialized to zero. Use init() or the individual setter functions to configure the filter before using update().

Definition at line 5 of file cf.h.

Here is the caller graph for this function:



9.1.2.2 ~CF()

```
template<typename T>
estimators::cf::CF< T >::~CF ()
```

Destroys the complementary filter.

Definition at line 10 of file cf.h.

9.1.3 Member Function Documentation

9.1.3.1 get_alpha()

```
template<typename T>
T estimators::cf::CF< T >::get_alpha ()
```

Gets the complementary filter coefficient.

Returns

Current complementary filter coefficient.

Definition at line 87 of file cf.h.

9.1.3.2 get_dt()

```
template<typename T>
T estimators::cf::CF< T >::get_dt ()
```

Gets the sampling time.

Returns

Current sampling time or control-loop period.

Definition at line 81 of file cf.h.

9.1.3.3 get_fc()

```
template<typename T>
T estimators::cf::CF< T >::get_fc ()
```

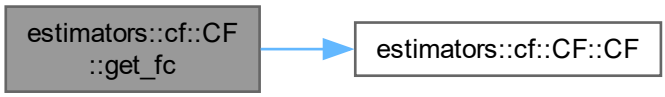
Gets the cutoff frequency.

Returns

Current cutoff frequency.

Definition at line 69 of file cf.h.

Here is the call graph for this function:



9.1.3.4 get_tau()

```
template<typename T>
T estimators::cf::CF< T >::get_tau ()
```

Gets the filter time constant.

Returns

Current filter time constant.

Definition at line 75 of file cf.h.

9.1.3.5 init()

```
template<typename T>
void estimators::cf::CF< T >::init (
            T fc_,
            T dt_)
```

Initializes the complementary filter.

Parameters

in	*fc_*	Cutoff frequency of the filter.
in	*dt_*	Sampling time or control-loop period.

This function initializes the filter parameters using the specified cutoff frequency and sampling time.

Definition at line 15 of file cf.h.

9.1.3.6 reset()

```
template<typename T>
void estimators::cf::CF< T >::reset ()
```

Resets the filter state.

Resets the internal state of the filter to its initial condition.

Definition at line 36 of file cf.h.

9.1.3.7 set_alpha()

```
template<typename T>
void estimators::cf::CF< T >::set_alpha (
            T alpha_)
```

Sets the complementary filter coefficient.

Parameters

in	*alpha_*	Complementary filter coefficient.

Definition at line 61 of file cf.h.

9.1.3.8 set_dt()

```
template<typename T>
void estimators::cf::CF< T >::set_dt (
            T dt_)
```

Sets the sampling time.

Parameters

in	*dt*↩ _↩	Sampling time or control-loop period.

Definition at line 55 of file cf.h.

9.1.3.9 set_fc()

```
template<typename T>
void estimators::cf::CF< T >::set_fc (
            T fc_)
```

Sets the cutoff frequency.

Parameters

in	*fc*↩ _↩	Cutoff frequency of the filter.

Definition at line 41 of file cf.h.

9.1.3.10 set_param()

```
template<typename T>
void estimators::cf::CF< T >::set_param (
            T fc_,
            T dt_)
```

Sets the filter parameters.

Parameters

in	*fc*↩ _↩	Cutoff frequency of the filter.
in	*dt*↩ _↩	Sampling time or control-loop period.

This function configures the cutoff frequency and sampling time of the complementary filter.

Definition at line 21 of file cf.h.

9.1.3.11 set_tau()

```
template<typename T>
void estimators::cf::CF< T >::set_tau (
            T tau_)
```

Sets the filter time constant.

Parameters

in	*tau_*	Time constant of the filter.

Definition at line 47 of file cf.h.

9.1.3.12 update()

```
template<typename T>
T estimators::cf::CF< T >::update (
            T x1_i,
            T x2_i)
```

Updates the complementary filter with new measurements.

Parameters

in	*x1_i*	First sensor measurement.
in	*x2_i*	Second sensor measurement.

Returns

Filtered or fused output.

Combines the two input measurements according to the complementary filter equation and updates the internal filter state.

Definition at line 30 of file cf.h.

9.1.4 Member Data Documentation

9.1.4.1 alpha

```
template<typename T>
T estimators::cf::CF< T >::alpha = 0.0  [private]
```

Complementary filter coefficient.

Definition at line 210 of file cf.h.

9.1.4.2 dt

```
template<typename T>
T estimators::cf::CF< T >::dt = 0.0  [private]
```

Sampling time or control-loop period.

Definition at line 205 of file cf.h.

9.1.4.3 fc

```
template<typename T>
T estimators::cf::CF< T >::fc = 0.0  [private]
```

Cutoff frequency of the filter.

Definition at line 195 of file cf.h.

9.1.4.4 tau

```
template<typename T>
T estimators::cf::CF< T >::tau = 0.0  [private]
```

Time constant of the filter.

Definition at line 200 of file cf.h.

The documentation for this class was generated from the following files:

- include/cf.h
- src/cf.tpp

Chapter 10

File Documentation

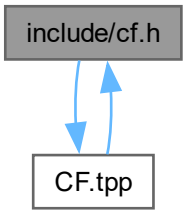
10.1 doc/mainpage.dox File Reference

10.2 include/cf.h File Reference

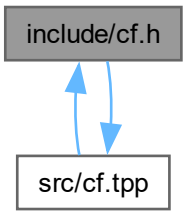
Declaration of the complementary filter class.

```
#include "CF.tpp"
```

Include dependency graph for cf.h:



This graph shows which files directly or indirectly include this file:



Classes

- class estimators::cf::CF< T >

  Generic complementary filter for sensor fusion.

Namespaces

- namespace cf

  Contains the complementary filter implementation.
- namespace estimators

  Contains estimation and sensor-fusion algorithms.
- namespace estimators::cf

10.2.1 Detailed Description

Declaration of the complementary filter class.

Author

Abhinay Kumar

This file defines the estimators::cf::CF class, which implements a complementary filter for sensor fusion.

A complementary filter combines measurements from different sensors by exploiting their complementary frequency characteristics. For example, a low-frequency measurement from one sensor can be combined with a high-frequency measurement from another sensor to obtain a more stable estimate.

The filter parameters are defined by the cutoff frequency, time step, time constant, and filter coefficient.

Version

1.0

Date

2026-08-11

Copyright

Definition in file cf.h.

10.3 cf.h

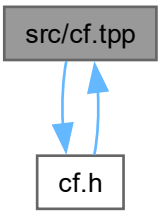
Go to the documentation of this file.

```
00001
00025
00026 #ifndef ESTIMATORS_COMPLIMENTARY_FILTER
00027 #define ESTIMATORS_COMPLIMENTARY_FILTER
00028
00029 namespace estimators
00030 {
00035
00036     namespace cf
00037     {
00042
00068         template <typename T>
00069         class CF
00070         {
00071         public:
00080             CF();
00081
00085             ~CF();
00086
00097             void init(T fc_, T dt_);
00098
00109             void set_param(T fc_, T dt_);
00110
00124             T update(T x1_i, T x2_i);
00125
00133             void reset();
00134
00140             void set_fc(T fc_);
00141
00147             void set_tau(T tau_);
00148
00154             void set_dt(T dt_);
00155
00161             void set_alpha(T alpha_);
00162
00168             T get_fc();
00169
00175             T get_tau();
00176
00182             T get_dt();
00183
00189             T get_alpha();
00190
00191         private:
00195             T fc = 0.0;
00196
00200             T tau = 0.0;
00201
00205             T dt = 0.0;
00206
00210             T alpha = 0.0;
00211         };
00212
00213 #include "CF.tpp"
00214
00215     } // namespace cf
00216
00217 } // namespace estimators
00218
00219 #endif // ESTIMATORS_COMPLIMENTARY_FILTER
```

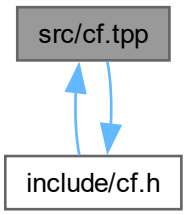
10.4 README.md File Reference

10.5 src/cf.tpp File Reference

`#include "cf.h"`
Include dependency graph for cf.tpp:



This graph shows which files directly or indirectly include this file:



10.6 cf.tpp

Go to the documentation of this file.

```
00001 #include "cf.h"
00002
00003 template <typename T>
00004 estimators::cf::CF<T>::CF()
00005 {
00006 }
00007
00008 template <typename T>
00009 estimators::cf::CF<T>::~CF()
00010 {
00011 }
00012
00013 template <typename T>
00014 void estimators::cf::CF<T>::init(T fc_, T dt_)
00015 {
```

```
00016     set_param(fc_, dt_);
00017 }
00018
00019 template <typename T>
00020 void estimators::cf::CF<T>::set_param(T fc_, T dt_)
00021 {
00022     fc = fc_;
00023     dt = dt_;
00024     tau = 1.0 / (6.2831853071795864769252867665590057 6 * fc);
00025     alpha = dt / (dt + tau);
00026 }
00027
00028 template <typename T>
00029 T estimators::cf::CF<T>::update(T x1_i, T x2_i)
00030 {
00031     return alpha * x1_i + (1.0 - alpha) * x2_i;
00032 }
00033
00034 template <typename T>
00035 void estimators::cf::CF<T>::reset()
00036 {
00037 }
00038
00039 template <typename T>
00040 void estimators::cf::CF<T>::set_fc(T fc_)
00041 {
00042     set_param(fc_, dt);
00043 }
00044
00045 template <typename T>
00046 void estimators::cf::CF<T>::set_tau(T tau_)
00047 {
00048     tau = tau_;
00049     fc = 1.0 / (6.2831853071795864769252867665590057 6 * tau);
00050     alpha = dt / (dt + tau);
00051 }
00052
00053 template <typename T>
00054 void estimators::cf::CF<T>::set_dt(T dt_)
00055 {
00056     set_param(fc, dt_);
00057 }
00058
00059 template <typename T>
00060 void estimators::cf::CF<T>::set_alpha(T alpha_)
00061 {
00062     alpha = alpha_;
00063     tau = ((1.0 - alpha) * dt) / alpha;
00064     fc = 1.0 / (6.2831853071795864769252867665590057 6 * tau);
00065 }
00066
00067 template <typename T>
00068 T estimators::cf::CF<T>::get_fc()
00069 {
00070     return fc;
00071 }
00072
00073 template <typename T>
00074 T estimators::cf::CF<T>::get_tau()
00075 {
00076     return tau;
00077 }
00078
00079 template <typename T>
00080 T estimators::cf::CF<T>::get_dt()
00081 {
00082     return dt;
00083 }
00084
00085 template <typename T>
00086 T estimators::cf::CF<T>::get_alpha()
00087 {
00088     return alpha;
00089 }
```

Index